

# Session: Convolutional Neural Networks

**Javier Serrano**

Applied Machine Learning  
Master in Data Science and Analytics



**Strathmore University**

*@iLabAfrica Centre*

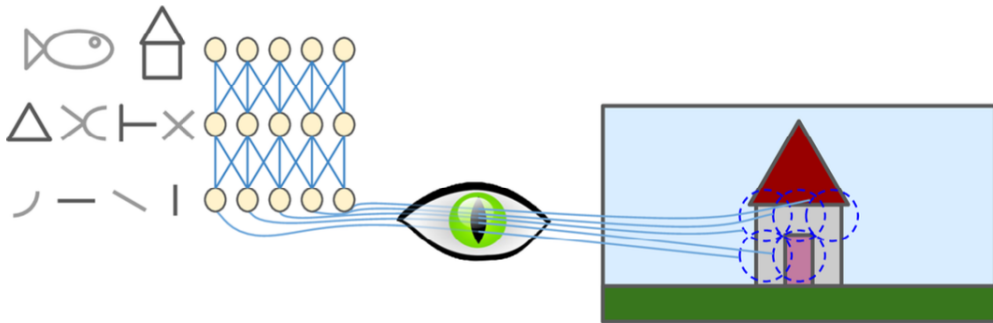
# ANN processing images

## Drawbacks

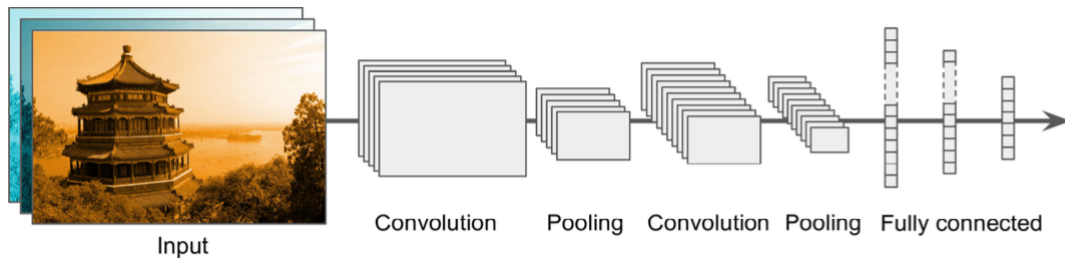
1. Too much computation
2. Treats local pixels same as pixel far apart
3. Sensitive to location of an object in an image

## How Humans Do It?

1. Biological neurons in the visual cortex respond to specific patterns in small regions of the visual field called receptive fields
2. As the visual signal makes its way through consecutive brain modules, neurons respond to more complex patterns in larger receptive fields.



## Typical CVV Architecture



# Summary

Convolutional Layers

Pooling Layers

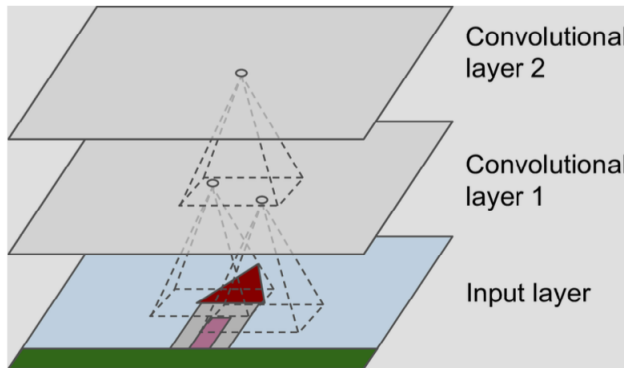
CNN Architectures

Typical problems in Computer Vision

## Convolutional Layers

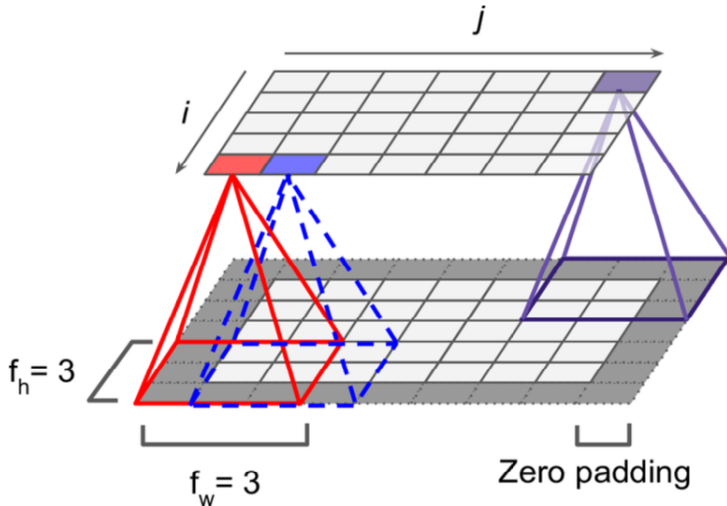
Same Idea than humans

- Convolutional layer are not connected to every single pixel in the input image but only to pixels in their receptive fields
- This architecture allows the network to concentrate on small low-level features in the first hidden layer, then assemble them into larger higher-level features in the next hidden layer



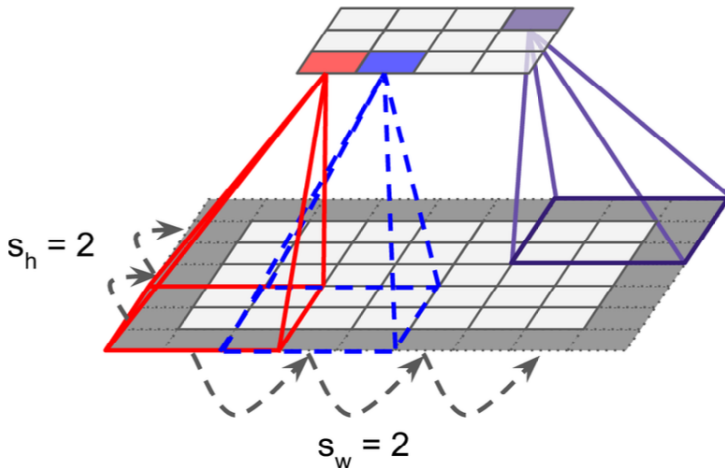
# Convolutional Layers

How does it work?



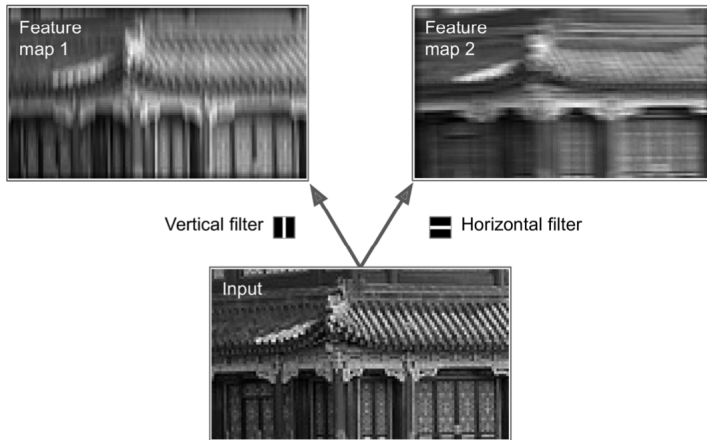
# Convolutional Layers

Reducing dimensionality using a stride of 2



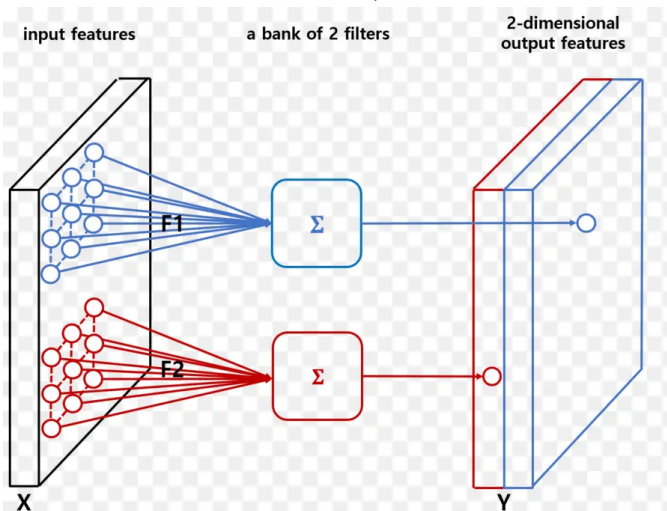
# Convolutional Layers

Applying two different filters to get two feature maps



# Convolutional Layers

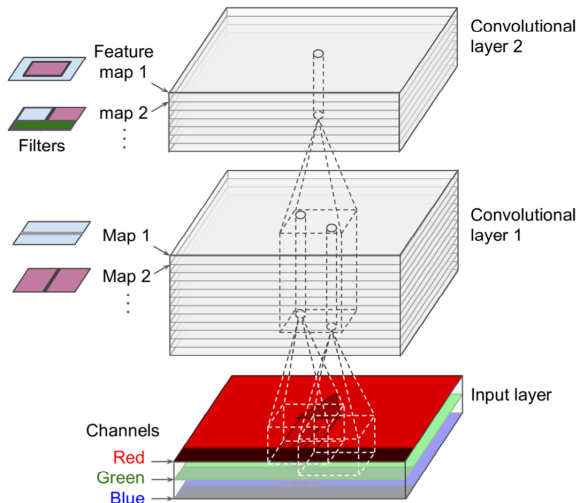
Multiple filters/kernels



## Convolutional Layers

### Stacking Multiple Feature Maps

- One neuron per pixel in each feature map
- All neurons within a given feature map share the same parameters
- A neuron's receptive field extends across all the previous layers' feature maps.



## Convolutional Layers

EXAMPLE: How many parameter we need?

Consider a CNN composed of three convolutional layers, each with  $3 \times 3$  kernels, a stride of 2, and "same" padding. The lowest layer outputs 100 feature maps, the middle one outputs 200, and the top one outputs 400. The input images are RGB images of  $200 \times 300$  pixels. What is the total number of parameters in the Convolutional layers?

1. its first convolutional layer has  $3 \times 3$  kernels, and the input has three channels (red, green, and blue), each feature map has  **$3 \times 3 \times 3$  weights, plus a bias term**. That's 28 parameters per feature map.
2. this first convolutional layer has 100 feature maps, it has a total of 2,800 parameters
3. The second convolutional layer has  $3 \times 3$  kernels and its input is the set of 100 feature maps of the previous layer, so each feature map has  **$3 \times 3 \times 100 = 900$  weights, plus a bias term**. Since it has 200 feature maps, this layer has  $901 \times 200 = 180,200$  parameters.

## Convolutional Layers

EXAMPLE: How many parameter we need? (cont.)

Consider a CNN composed of three convolutional layers, each with  $3 \times 3$  kernels, a stride of 2, and "same" padding. The lowest layer outputs 100 feature maps, the middle one outputs 200, and the top one outputs 400. The input images are RGB images of  $200 \times 300$  pixels. What is the total number of parameters in the Convolutional layers?

- 4 the third and last convolutional layer also has  $3 \times 3$  kernels, and its input is the set of 200 feature maps of the previous layers, so each feature map has  $3 \times 3 \times 200 = 1,800$  weights, plus a bias term. Since it has 400 feature maps, this layer has a total of  $1,801 \times 400 = 720,400$  parameters
- 5 the CNN has  $2,800 + 180,200 + 720,400 = 903,400$  parameters.

## Convolutional Layers

EXAMPLE: How much RAM memory we need?

Consider a CNN composed of three convolutional layers, each with  $3 \times 3$  kernels, a stride of 2, and "same" padding. The lowest layer outputs 100 feature maps, the middle one outputs 200, and the top one outputs 400. The input images are RGB images of  $200 \times 300$  pixels. How much RAM memory we need to train the network? And for inference

1. Since we are using a stride of 2 and "same" padding, as the input channels are  $200 \times 300$  pixels, the first layer's feature maps are  $100 \times 150$ , the second layer's feature maps are  $50 \times 75$ , and the third layer's feature maps are  $25 \times 38$
2. the first convolutional layer has 100 feature maps, this first layer takes up  $4 \times 100 \times 150 \times 100 = 6$  million bytes (6 MB)
3. The second layer takes up  $4 \times 50 \times 75 \times 200 = 3$  million bytes (3 MB)
4. the third layer takes up  $4 \times 25 \times 38 \times 400 = 1,520,000$  bytes (about 1.5 MB)
5. The 903,400 parameters: 3,613,600 bytes (about 3.6 MB)

# Convolutional Layers

EXAMPLE: How much RAM memory we need? (cont.)

Consider a CNN composed of three convolutional layers, each with  $3 \times 3$  kernels, a stride of 2, and "same" padding. The lowest layer outputs 100 feature maps, the middle one outputs 200, and the top one outputs 400. The input images are RGB images of  $200 \times 300$  pixels. How much RAM memory we need to train the network? And for inference

- **Total Inference:** 12,613,600 bytes ( $6 + 3 + 3.6 = 12.6$  MB).
- **Total Training** mini-batch of 50 images):
  1. the input images, which is  $50 \times 4 \times 200 \times 300 \times 3 = 36$  million bytes (36 MB)
  2. the three layers are  $6 + 3 + 1.5 = 10.5$ , so for 50 instances it is 525 MB
  3. The parameters 3.6 MB
  4. TOTAL (optimistic):  $36 + 525 + 3.6 = 564.6$  MB

# Summary

Convolutional Layers

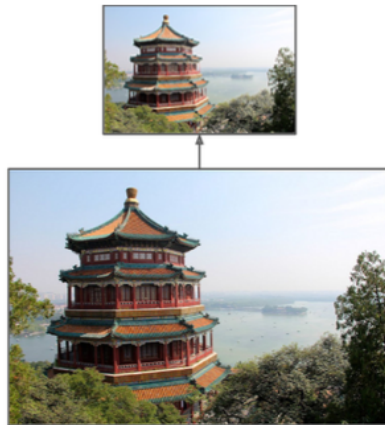
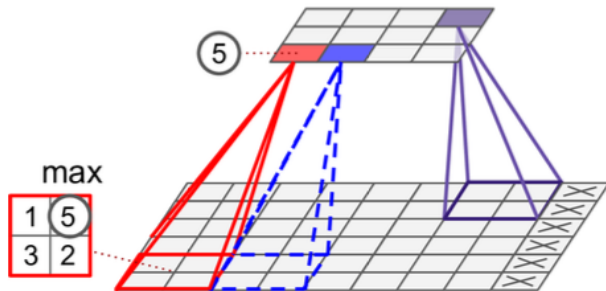
Pooling Layers

CNN Architectures

Typical problems in Computer Vision

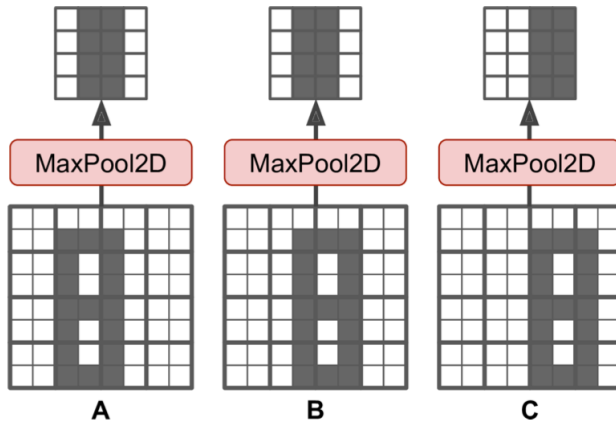
# Pooling Layers

Subsampling: Max Pooling Layer ( $2 \times 2$  pooling kernel, stride 2, no padding)



## Pooling Layers

Another advantage: Invariance to small translations



Invariance is not always desirable (Semantic segmentation)

# Summary

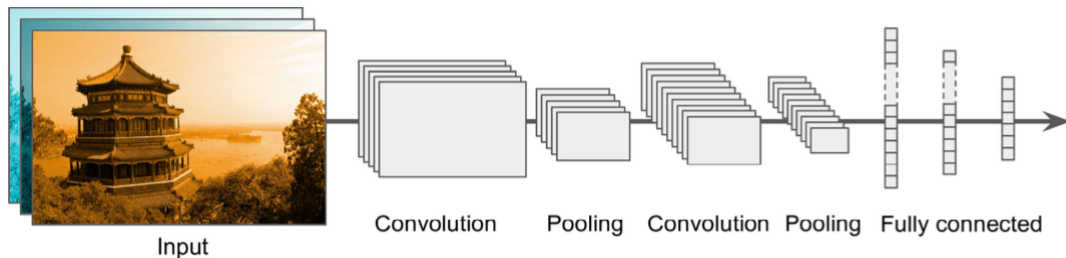
Convolutional Layers

Pooling Layers

CNN Architectures

Typical problems in Computer Vision

## CNN Architectures



- After every convolutional layer comes a ReLu activation function.
- The fully connected layers also use ReLu

# CNN Architectures

## A simple CNN Implementation

```
model = keras.models.Sequential([
    keras.layers.Conv2D(64, 7, activation="relu", padding="same",
                        input_shape=[28, 28, 1]),
    keras.layers.MaxPooling2D(2),
    keras.layers.Conv2D(128, 3, activation="relu", padding="same"),
    keras.layers.Conv2D(128, 3, activation="relu", padding="same"),
    keras.layers.MaxPooling2D(2),
    keras.layers.Conv2D(256, 3, activation="relu", padding="same"),
    keras.layers.Conv2D(256, 3, activation="relu", padding="same"),
    keras.layers.MaxPooling2D(2),
    keras.layers.Flatten(),
    keras.layers.Dense(128, activation="relu"),
    keras.layers.Dropout(0.5),
    keras.layers.Dense(64, activation="relu"),
    keras.layers.Dropout(0.5),
    keras.layers.Dense(10, activation="softmax")
])
```

# CNN Architectures

LeNet-5 (LeCun, 1998)

| Layer | Type            | Maps | Size           | Kernel size  | Stride | Activation |
|-------|-----------------|------|----------------|--------------|--------|------------|
| Out   | Fully connected | –    | 10             | –            | –      | RBF        |
| F6    | Fully connected | –    | 84             | –            | –      | tanh       |
| C5    | Convolution     | 120  | $1 \times 1$   | $5 \times 5$ | 1      | tanh       |
| S4    | Avg pooling     | 16   | $5 \times 5$   | $2 \times 2$ | 2      | tanh       |
| C3    | Convolution     | 16   | $10 \times 10$ | $5 \times 5$ | 1      | tanh       |
| S2    | Avg pooling     | 6    | $14 \times 14$ | $2 \times 2$ | 2      | tanh       |
| C1    | Convolution     | 6    | $28 \times 28$ | $5 \times 5$ | 1      | tanh       |
| In    | Input           | 1    | $32 \times 32$ | –            | –      | –          |

- Initially used with MNIST images of  $28 \times 28$  pixels

# CNN Architectures

## AlexNet (2012)

- It won the 2012 ImageNet ILSVRC challenge (Difficult problem)
- The images are large (256 pixels high) and there are 1,000 classes
- It is much larger and deeper than LeNet-5
- It stacks convolutional layers directly on top of each other, instead of stacking a pooling layer on top of each convolutional layer
- Use ReLu as activation function
- Use Dropout in the fully connected layers
- Perform **Data augmentation**

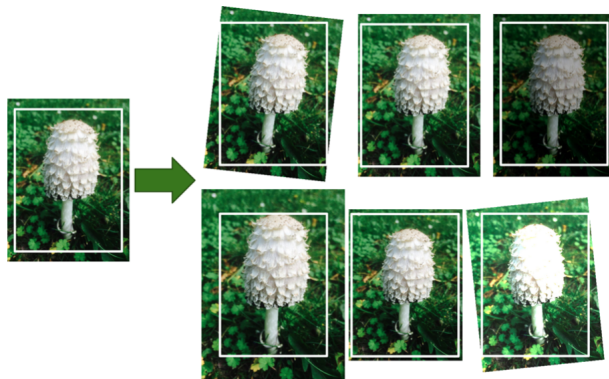
# CNN Architectures

## AlexNet Architecture

| Layer | Type            | Maps    | Size             | Kernel size    | Stride | Padding | Activation |
|-------|-----------------|---------|------------------|----------------|--------|---------|------------|
| Out   | Fully connected | —       | 1,000            | —              | —      | —       | Softmax    |
| F10   | Fully connected | —       | 4,096            | —              | —      | —       | ReLU       |
| F9    | Fully connected | —       | 4,096            | —              | —      | —       | ReLU       |
| S8    | Max pooling     | 256     | $6 \times 6$     | $3 \times 3$   | 2      | valid   | —          |
| C7    | Convolution     | 256     | $13 \times 13$   | $3 \times 3$   | 1      | same    | ReLU       |
| C6    | Convolution     | 384     | $13 \times 13$   | $3 \times 3$   | 1      | same    | ReLU       |
| C5    | Convolution     | 384     | $13 \times 13$   | $3 \times 3$   | 1      | same    | ReLU       |
| S4    | Max pooling     | 256     | $13 \times 13$   | $3 \times 3$   | 2      | valid   | —          |
| C3    | Convolution     | 256     | $27 \times 27$   | $5 \times 5$   | 1      | same    | ReLU       |
| S2    | Max pooling     | 96      | $27 \times 27$   | $3 \times 3$   | 2      | valid   | —          |
| C1    | Convolution     | 96      | $55 \times 55$   | $11 \times 11$ | 4      | valid   | ReLU       |
| In    | Input           | 3 (RGB) | $227 \times 227$ | —              | —      | —       | —          |

# CNN Architectures

## Data Augmentation

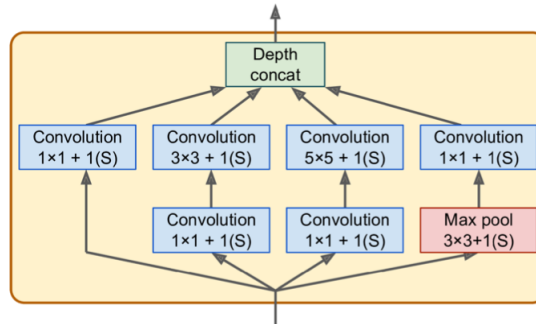


Reduce overfitting & Produce more instances for training

# CNN Architectures

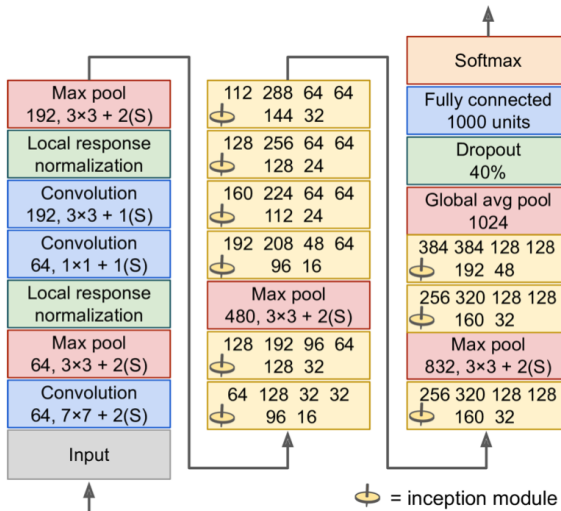
## GoogLeNet (2014)

- It won the 2014 ImageNet ILSVRC challenge
- The main innovation was the **inception modules**, which make it possible to have a much deeper net with fewer parameters.



# CNN Architectures

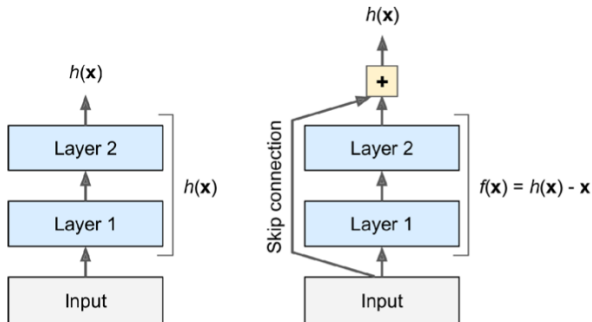
## GoogLeNet Architecture



# CNN Architectures

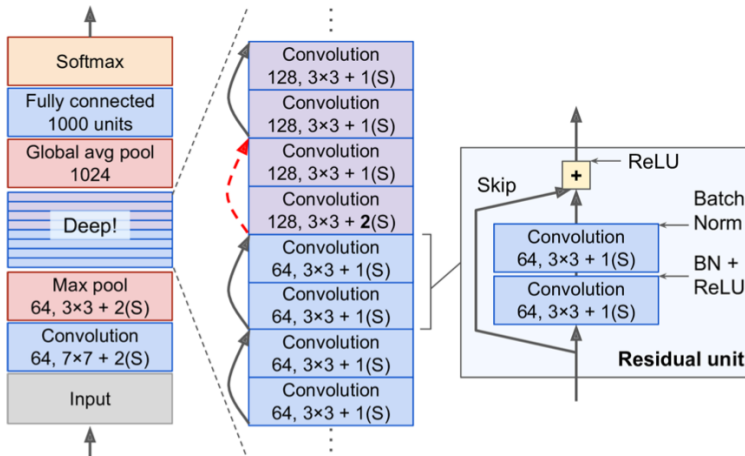
## ResNet (2015)

- It won the 2015 ImageNet ILSVRC challenge using an extremely deep CNN: 152
- The main innovation was the introduction of skip connections
- Residual learning



# CNN Architectures

## ResNet Architecture



# Summary

Convolutional Layers

Pooling Layers

CNN Architectures

Typical problems in Computer Vision

## Typical problems in Computer Vision

- Classification of images
- Localization of and object in an image
- Object detection
- Semantic segmentation

# Typical problems in Computer Vision

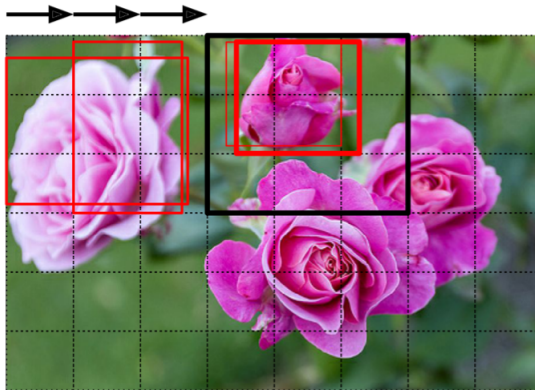
## Localization of and object in a picture

- The problem can be expressed as a **regression task**
- to predict a bounding box: coordinates of the object's center, height and width.
- So, we **predict 4 numbers**
- it can be trained using the MSE loss or IoU.
- Getting the labels for the bounding boxes (hard and costly)

# Typical problems in Computer Vision

## Object Detection

- Object detection is the task of classifying and localizing multiple objects in an image.
- Initial solution: sliding a CNN across the image
- Post-process to remove detections of the same object



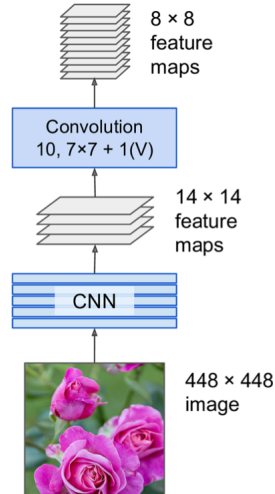
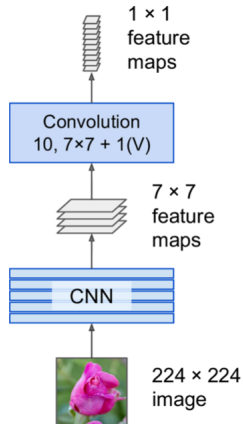
# Typical problems in Computer Vision

## Object Detection: Fully Convolutional Networks (FCN)

- The idea is replace the dense layers at the top of a CNN by convolutional layers
- FCNs can efficiently process images of any width and height.
- They are most useful for object detection and semantic segmentation because they only need to look at the image once
- We can convert dense layers to convolutional layers to create an FCN as follow:
  1. Replace the lowest dense layer with a convolutional layer with a kernel size equal to the layer's input size, with one filter per neuron in the dense layer, and using "valid" padding.
  2. The stride should be 1
  3. The activation function should be the same as the dense layer's.
  4. The other dense layers should be converted the same way, but using  $1 \times 1$  filters.
  5. It is actually possible to convert a trained CNN this way by appropriately reshaping the dense layers' weight matrices.

# Fully Convolutional Networks (FCN)

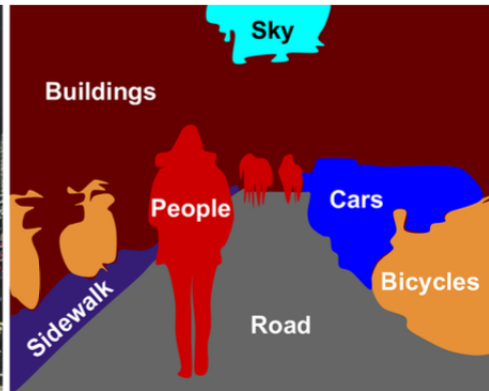
The same fully convolutional network processing a small and a large image



# Typical problems in Computer Vision

## Semantic Segmentation

- Each pixel is classified according to the class of the object it belongs to



# Typical problems in Computer Vision

Semantic Segmentation is a difficult problem

- Main difficulty in this task is they gradually lose their spatial resolution when progress through the CNN
- This spatial information needs to be restored somehow to accurately predict the class of each pixel.
- Many different approaches to tackle this problem

# Summary

Convolutional Layers

Pooling Layers

CNN Architectures

Typical problems in Computer Vision

## References

- A. Geron, *Hands-on Machine Learning with Scikit-Learn, Keras, and TensorFlow*, O'Reilly, 2019.
- Andrew Ng course on Deep Learning  
<https://www.youtube.com/watch?v=E13qqHb3J7U>
- I. Goodfellow, Y. Bengio, A. Courville, *Deep Learning*, MIT Press, 2016.  
<https://github.com/janishar/mit-deep-learning-book-pdf>
- Simon J.D. Prince *Understanding Deep Learning* 2024  
<https://udlbook.github.io/udlbook/>